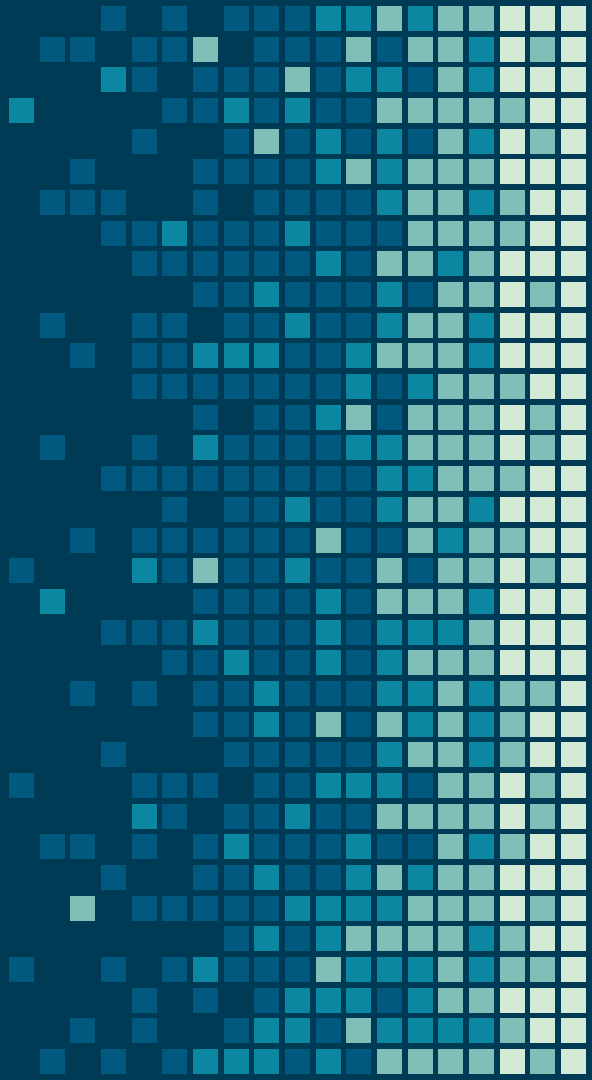
EMBEDDED CONTROL

Prof. Luca De Cicco
luca.decicco@poliba.it
https://c3lab.poliba.it/LDC



Stepper motors

- A stepper motor moves in discrete angular increments
- Position is usually controlled by counting commanded steps (open-loop mode)
- Steppers are used in a variety of applications:
  - 3D printers
  - CNC machines (up to 5 axes, rotary and linear)
- In this lecture: we focus on how to generate motion commands from the microcontroller



Stepper motors



Steppers basics

- Stepper drivers convert commands into phase currents
- With `STEP/DIR` drivers, each `STEP` pulse advances the motor by one step increment in a given direction `DIR` (cw or ccw)
- **Stepper resolution** (step angle): angular displacement per full step (°/step). Typical values: 1.8 °/step
- **Steps per revolution**: how many full steps are required to make the shaft rotate a full revolution.

```
STEP_PER_REV = 360°/STEPPER_RESOLUTION
```

Typical value: (360°/rev) /(1.8°/step) = 200 step/rev

Microstepping

- Microstepping divides each full step into smaller commanded increments
- Driver configuration selects the microstep factor *M*: 2, 4, 8, 16, 32, …
- Effective steps per revolution: $N_{steps/rev} = N_{full\text{-}steps/rev} \times M$

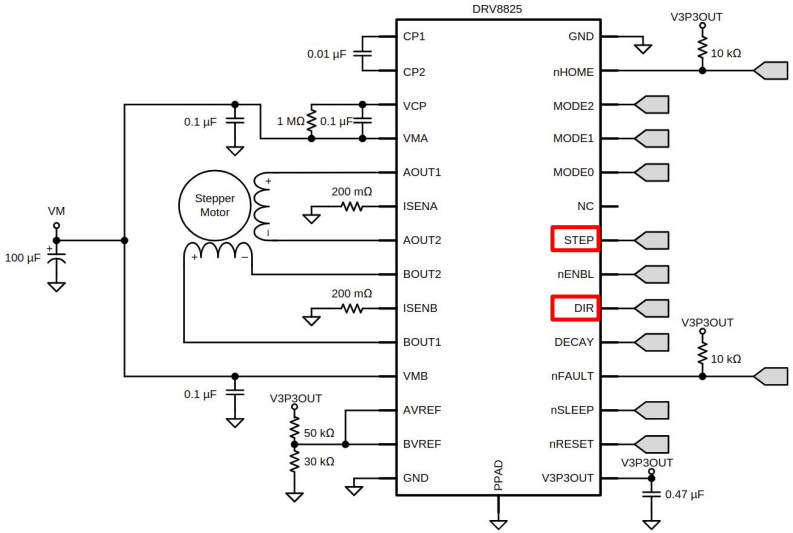
Pros

- finer command resolution (better precision)
- smoother motion
- reduced vibrations/resonance

Cons

- more pulses for the same displacement
- higher required timer frequency

Drivers



Drivers have typically two inputs:

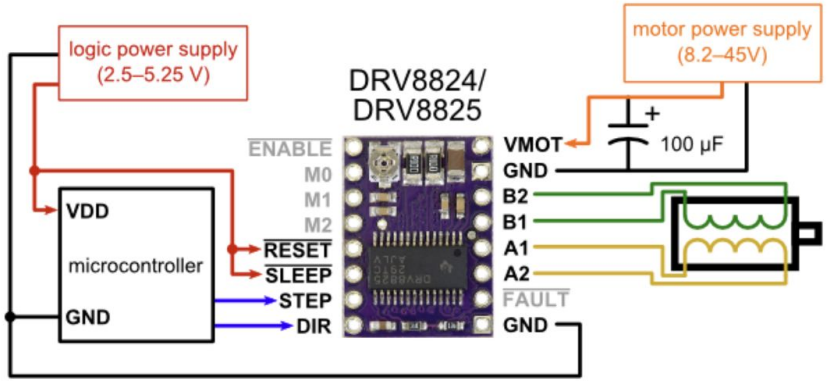
- DIR: sets the rotation direction (f.i. `LOW`=clockwise, `HIGH`=counter clockwise)
- `STEP`: is and input to generate steps at the stepper shaft

Wiring

- `DIR` pin is connected to a `GPIO` output
- `STEP` pin is connected to a PWM Timer single channel

Example: DRV8825 driver break out board

- MS1,MS2,MS3 regulate the microstepping (see table)
- VMOT-GND will be connected to the Motor power supply (max 35V)
- 1A,1B,2A,2B are connected to the phases of the stepper
- VDD-GND are the logic voltage supply (3.3V)



MODE0	MODE1	MODE2	Microstep Resolution
Low	Low	Low	Full step
High	Low	Low	Half step
Low	High	Low	1/4 step
High	High	Low	1/8 step
Low	Low	High	1/16 step
High	Low	High	1/32 step
Low	High	High	1/32 step
High	High	High	1/32 step

Linear motion

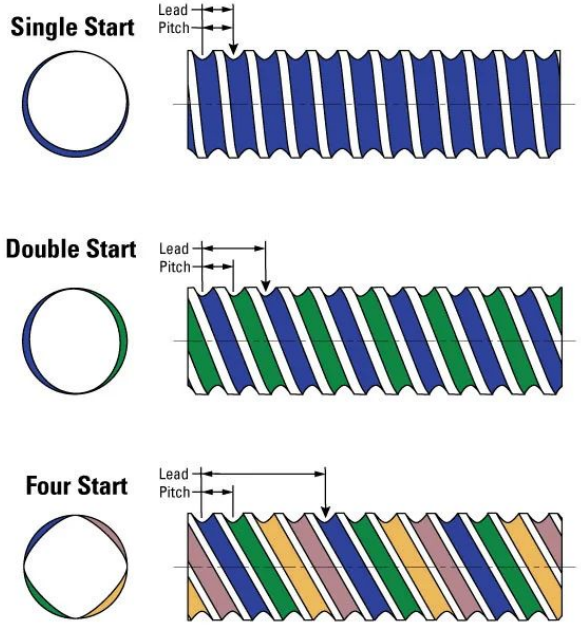
The motor shaft is connected to an axis that we want to move:

- Linear axis. Mechanical transmission is needed to convert rotation into translation
- To generate linear motion we can use:
  - leadscrews/ballscrews of a given pitch
  - belts and pulley
  - rack and pinion



Leadscrews: pitch, lead, and displacement per revolution

- Multi-start lead screws: multiple *independent* intertwined helical threads (*vite a filettatura multipla/multiprincipio*)
- Single-start: one helical thread
- Pitch *P*: distance between adjacent threads
- Lead *L*: linear displacement for one full revolution of the shaft
- Single-start screw: `L = P`
- Multiple-start screw: `L = n_starts * P`



Source

From motor steps to axis displacement

- **Step scale** (S): number of steps to obtain a given unit of displacement (linear angular) of the axis
- Step scale also consider transmission ratio G (G=1 direct coupling, G>1 reduction)

- Rotary axis: $S = N_{\text{full-steps/rev}} * M * G / 360°$
- Linear axis: $S = N_{\text{full-steps/rev}} * M * G / L$

where L is the lead and M is the microstepping factor.

Example

Calculate the step scale in the case:

- Stepper with 200 step/rev
- 16 micro steps
- The stepper is coupled with a single-start lead screw having a pitch P=2mm

Solution: since we have a single-start lead screw L=P=2 mm
Then:

$$S = 200 * 16/2 = 1600 \text{ steps/mm}$$

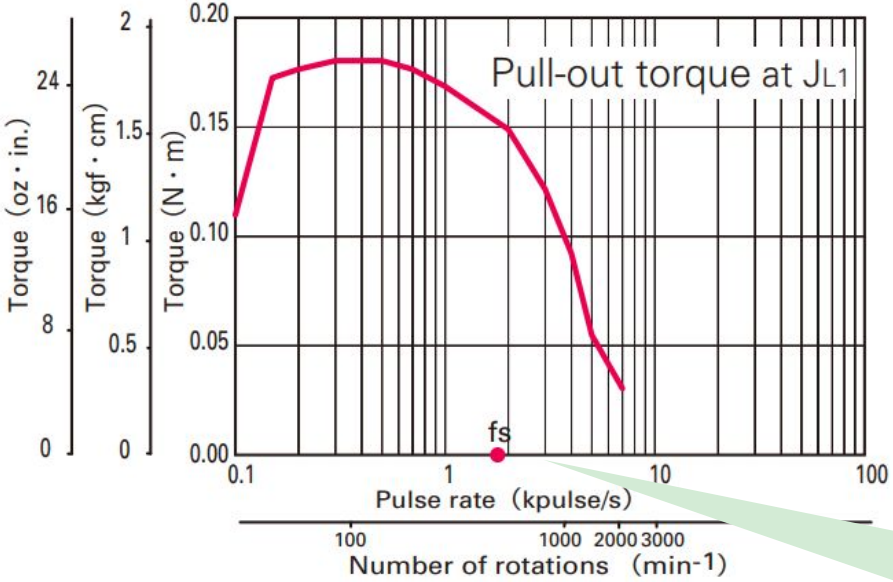
Using a stepper for position control

- We want to move a given axis of an amount `DISTANCE`
- Knowing the step scale we know how many steps we have to generate:

  `STEPS = S` (step/mm) `* DISTANCE` (mm)

- The speed depends on the frequency at which we generate steps: `v = f`$_{step}$`/S`
- Example: to move at `v=10`mm/s with `S=1600` steps/mm (ex,. prev slide) we have: f_{step}`=1600*10=16000` steps/mm
- Notice: the faster we go, the lower the torque

Typical Torque vs steps/s curve



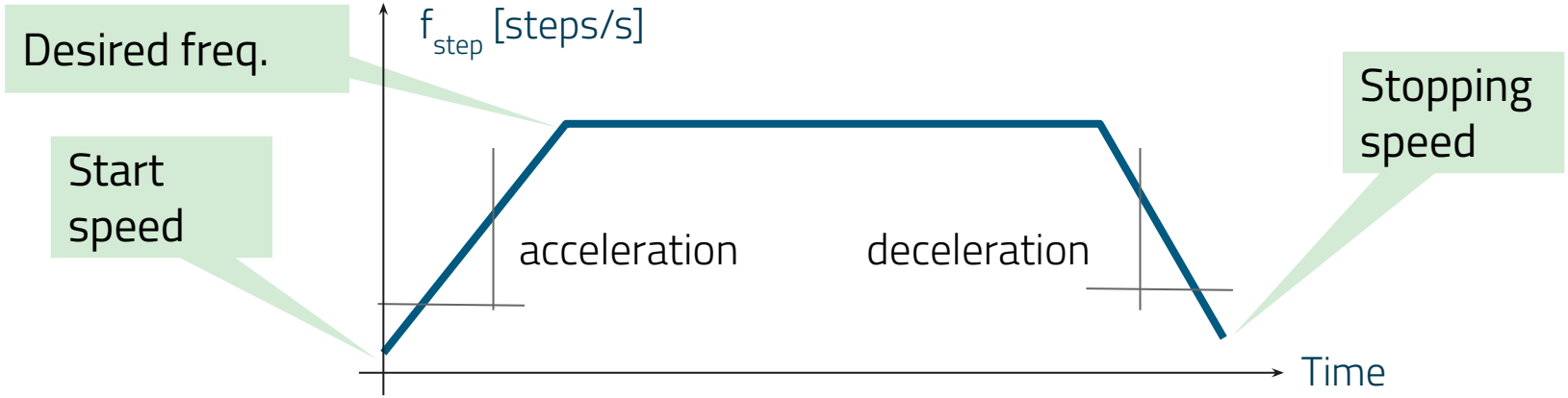
Constant current circuit
Source voltage : DC24V · operating current : 1A/phase,
2-phase energization (full-step)

- Pull-out torque: max torque at speed if already running
- f_s: maximum frequency at which the motor can start reliably under given load

Starting the stepper with a step frequency higher than the starting frequency f_s may result in missing steps!

Acceleration Profiles to avoid missed steps

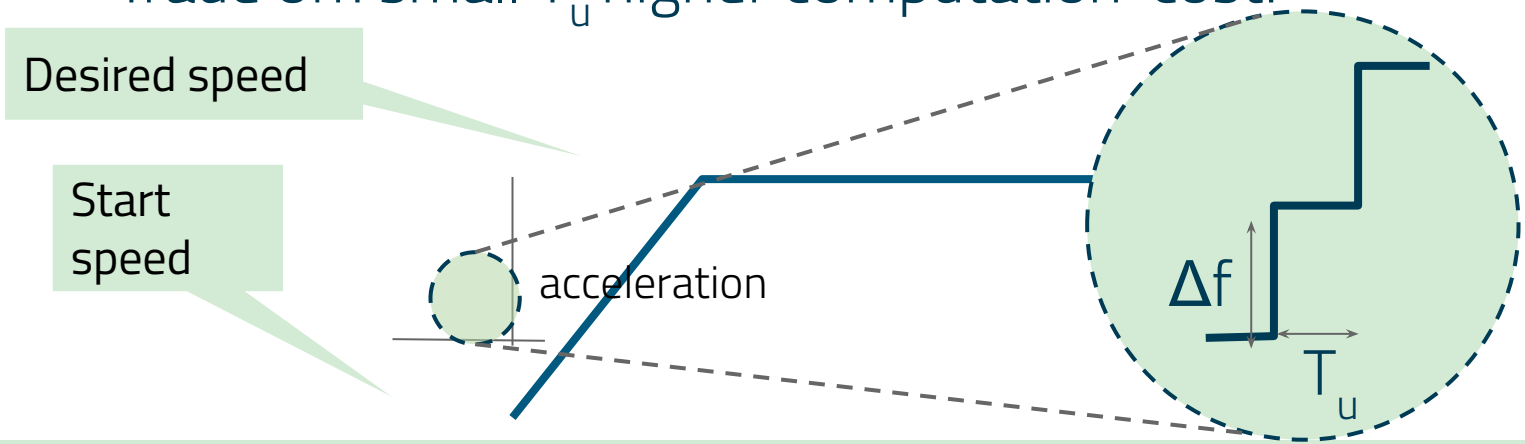
- Start with a step frequency below f_s and increase the frequency up to the desired frequency (acceleration profile)
- When stopping we should decrease the speed at a given pace (deceleration profile)
- A timer is used, changing its frequency: $f_{step} = \frac{f_{timer}}{(PSC+1)(ARR+1)}$



Implementing acceleration profiles [1]

Design choices:

- The acceleration and deceleration a (in step/s^2)
- The update interval T_u
- frequency increment: $\Delta f = a\, T_u$
- Trade off: small T_u higher computation cost!



Position control with STEP pulses

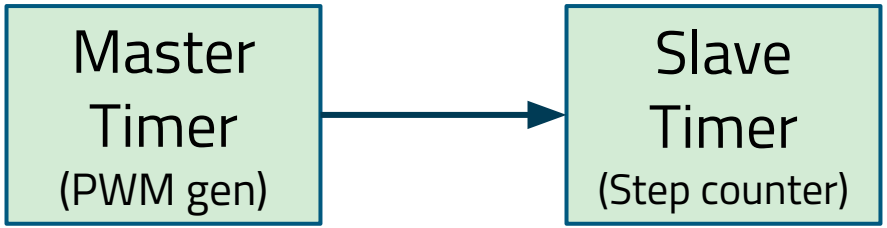
Problem: you want to move an axis by a given displacement (linear or angular)

Solution: you have to generate an appropriate number of steps (recall `N_STEPS = STEP_SCALE` (step/mm) `* DISTANCE` (mm))

- Set `DIR` according to the sign of displacement
- Generate exactly `N_STEPS` pulses on `STEP`
- Stop the PWM after the last pulse
- Can be done in several ways. One of the best approaches is using timers chaining

Chaining Timers

- A timer can serve as a clock to another timer, this is called chaining
- The master generates the pulses, the slave reads the pulses and counts them
- We can set the ARR of the slave timer to generate an interrupt when the required steps have been counted (this is all hardware)



Chaining Timers: example TIM2 master

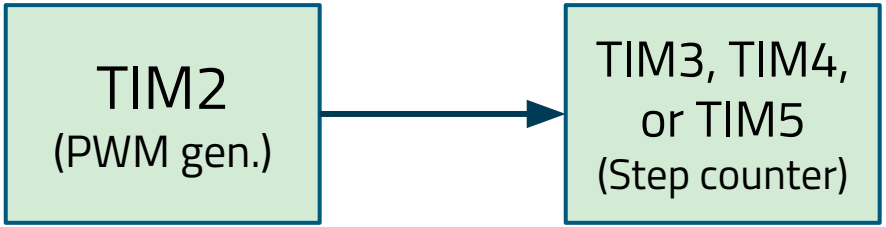
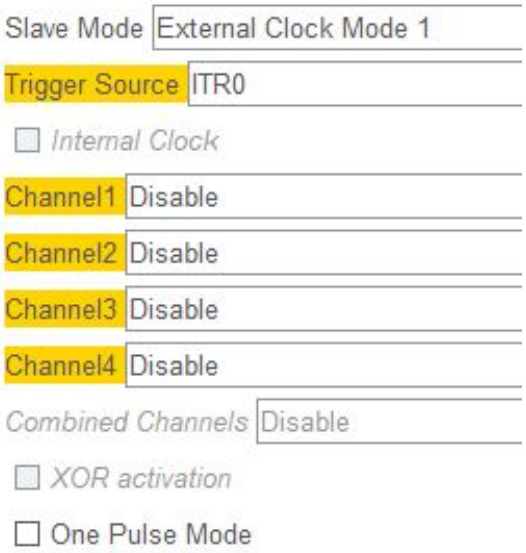


Table 112. TIMx internal trigger connections

Slave TIM	ITR0 (TS = 000)	ITR1 (TS = 001)	ITR2 (TS = 010)	ITR3 (TS = 011)
TIM2	TIM1	TIM8	TIM3	TIM4
TIM3	TIM1	TIM2	TIM5	TIM4
TIM4	TIM1	TIM2	TIM3	TIM8
TIM5	TIM2	TIM3	TIM4	TIM8

If `TIM2` is the master, `TIM3/TIM4/TIM5` can receive `TIM2` through the highlighted `ITR` lines

Chaining timers in CubeMX



- Select the slave timer
- Set Slave Mode = External Clock Mode 1
- Set Trigger Source = ITRx corresponding to the master timer
- Enable update interrupt
- Set ARR = N_steps - 1 (This should be done programmatically in the firmware)
- The correct ITR number depends on the timer pair and on the STM32 family. Always check the reference manual.

Controlling a Nema stepper

Goal

- We use a Nema 17 with a 1.8 deg resolution (200 seps/rev)
- The employed driver is the Texas Instruments DRV8825

1st exercise

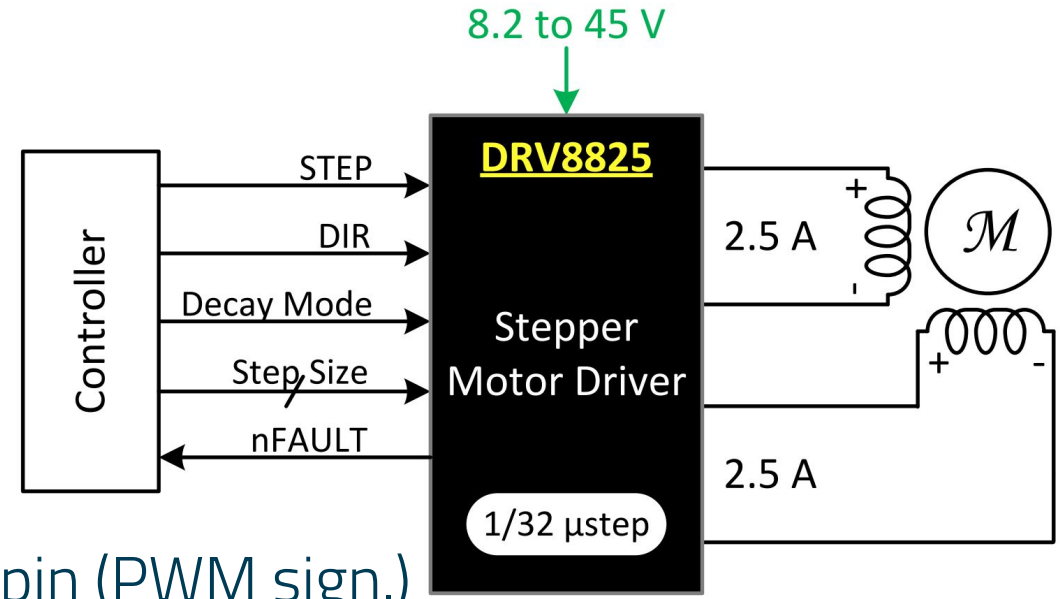
- We want to implement velocity control

2nd exercise

- We want to implement position control

The driver

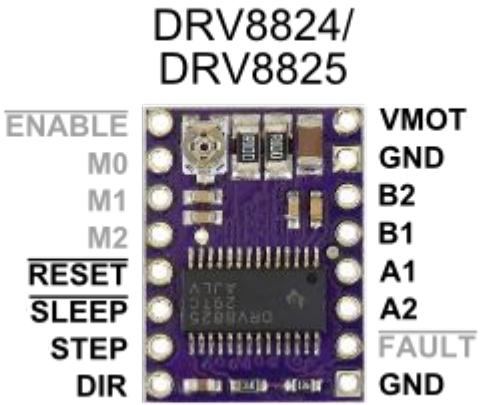
- Supports up to 45V motors (check before the rating of your Nema)
- Up to 1/32 Microst.
- Fault condition indication (nFAULT)
- Driven through STEP pin (PWM sign.)
- DIR pin to set the rotation direction



Microstepping settings

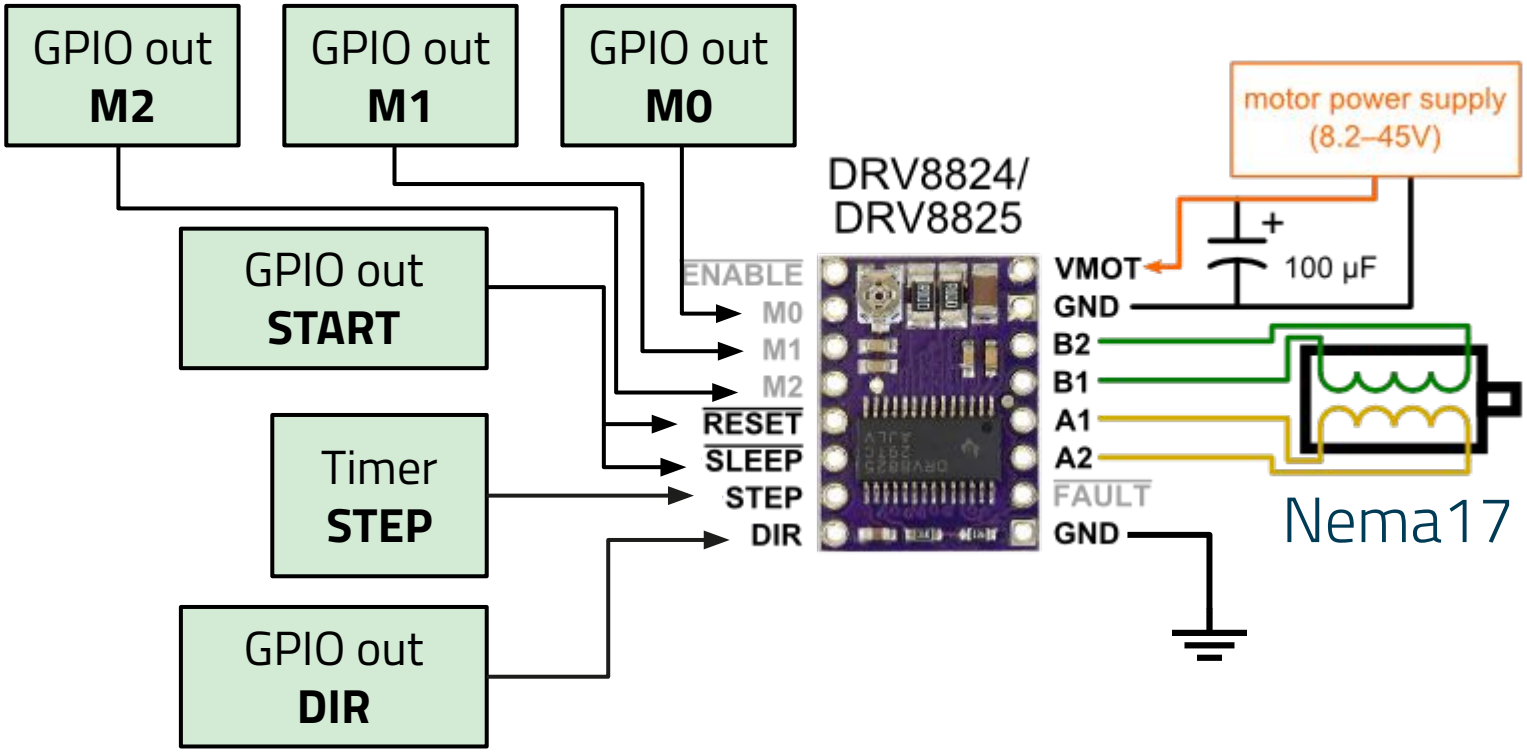
Microstepping is set using the `M0`, `M1`, `M2` pins.

Either wire those pins to GND/VDD or use GPIO outputs to have higher flexibility



MODE0	MODE1	MODE2	Microstep Resolution
Low	Low	Low	Full step
High	Low	Low	Half step
Low	High	Low	1/4 step
High	High	Low	1/8 step
Low	Low	High	1/16 step
High	Low	High	1/32 step
Low	High	High	1/32 step
High	High	High	1/32 step

Driver breakout board wiring



Wiring

- The Nucleo board will be powered externally by the power supply through the VIN pin
- The motor will be driven at 12V
- We will experiment with different microstep settings (M0, M1, M2)

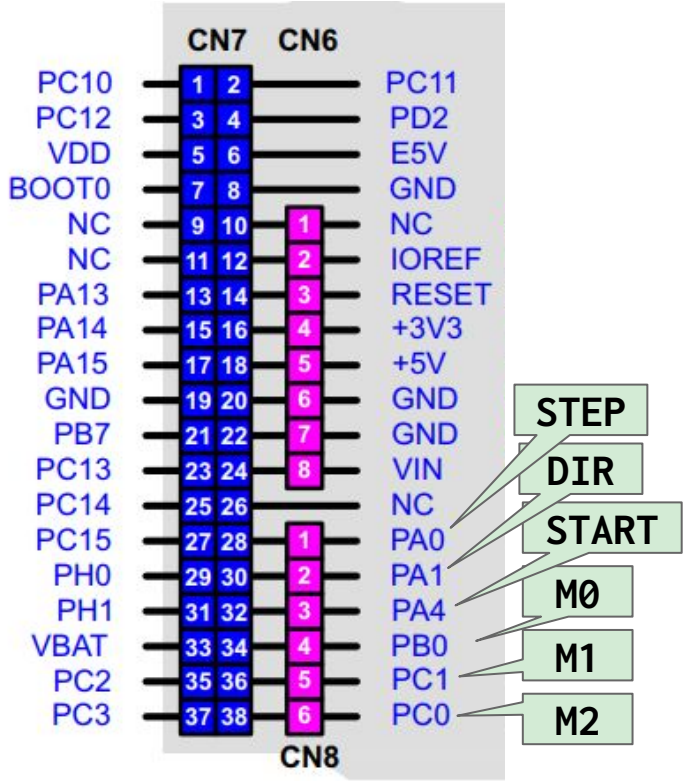
© Luca De Cicco - Embedded Control - Politecnico di Bari

Project configuration

We need:

- A Timer for PWM generation named **STEP**. The duty cycle will always be set to 50%. The frequency (ARR, PSC) will affect the motor speed.
- A GPIO output that named **DIR** will be used to change the direction of rotation
- A GPIO output named **START** will be used to start/stop the motor
- Three GPIO output named **M0**, **M1**, **M2** to set microstepping

Driver breakout board wiring



GPIO output pins

PA1 → DIR

PA4 → START

PB0 → M0

PC1 → M1

PC0 → M2

Pin...	...	GPIO out...	...	GPIO Pull-up/Pu...	Maxim...	User Label
PA1	...	Low	...	No pull-up and ...	Low	DIR
PA4	...	Low	...	No pull-up and ...	Low	START
PB0	...	Low	...	No pull-up and ...	Low	M0
PC0	...	Low	...	No pull-up and ...	Low	M2
PC1	...	Low	...	No pull-up and ...	Low	M1
PC13	...	n/a	...	No pull-up and ...	n/a	B1 [Blue Pus...

Timer

PA0 → TIM2_CH1 (32 bit PWM, APB1)

Slave Mode	Disable
Trigger Source	Disable
Clock Source	Disable
Channel1	PWM Generation CH1
Channel2	Disable

Setting the PWM freq. function of angular vel.

$$f_PWM = f_CLK/((ARR+1)(PSC+1))$$

What's the PWM frequency (microsteps/second) that we have to implement to reach a given desired rev/s speed?

$$f_PWM\ (microstep/s) = K * vel\ (rev/s)$$

where $K = STEPS_PER_REV*MICROSTEPS\ (steps/rev)$

Bare minimum implementation (1)

A not reusable implementation (**don't do this!**)

In the main initialize the microstep (32 in this example) and make sure the driver is in sleep mode:

```
HAL_GPIO_WritePin(M0_GPIO_Port, M0_Pin, GPIO_PIN_SET);
HAL_GPIO_WritePin(M1_GPIO_Port, M1_Pin, GPIO_PIN_SET);
HAL_GPIO_WritePin(M2_GPIO_Port, M2_Pin, GPIO_PIN_SET);
HAL_GPIO_WritePin(START_GPIO_Port, START_Pin, GPIO_PIN_RESET);
```

We want the nema to start rotating when pressing the blue button in its exception handler.

Bare minimum implementation (2)

Handler starts/stops the PWM to drive the nema when B1 button is pressed

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
     if (GPIO_Pin == B1_Pin) {
          if (HAL_GPIO_ReadPin(START_GPIO_Port, START_Pin) == GPIO_PIN_SET) {
               HAL_GPIO_WritePin(START_GPIO_Port, START_Pin, GPIO_PIN_RESET);
               set_rpm(0);
               HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_1);
          } else {
               HAL_GPIO_WritePin(START_GPIO_Port, START_Pin, GPIO_PIN_SET);
               set_rpm(120.0);
               HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
          }
     }
}
```

Bare minimum implementation (3)

Sets the ARR/CCR values

```
#define STEPS_PER_REV ((uint32_t)200)
#define MICROSTEPS    ((uint32_t)32)
#define TIMER_CLK     ((uint32_t)84000000)

void set_rpm(float rpm) {
      uint32_t arr,psc,ccr;
      float f_pwm;
      GPIO_Pin_State dir;
      dir = ( rpm > 0.0f ? GPIO_PIN_RESET : GPIO_PIN_SET );   /* Direction function of rpm sign */
      HAL_GPIO_WritePin(DIR_GPIO_PORT, DIR_PIN, dir);         /* Set direction */
      psc = 0;                                                /* Not always ok! how to fix? */
      f_pwm = STEPS_PER_REV*MICROSTEPS* (fabsf(rpm)/60.0);    /* Microsteps frequency [Hz] */
      arr = (float)TIMER_CLK/f_pwm/(psc+1) - 1;               /* ARR */
      ccr = arr >> 1;                                         /* ccr = arr/2; 50% duty cycle */
      __HAL_TIM_SET_AUTORELOAD(&htim2, arr);                  /* Set ARR */
      __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, ccr);      /* Set CCR */
      htim2.Instance->EGR = TIM_EGR_UG;
}
```

What's wrong with this implementation?

- We cannot control more than one nema motor without duplicating the code!
- We cannot set programmatically the microsteps -> we should implement a function to do this
- Solution: implement a "nema" structure, a "driver" structure, a "nema-driver" structure composing the two structures

Sketch of the structures

```
typedef struct {
      uint16_t steps_per_rev;      /* Steps per revolution: ex. 200 */
      GPIO_TypeDef *dir_port;      /* DIR gpio port */
      uint16_t dir_pin;            /* DIR gpio pin */
      TIM_HandleTypeDef *step_tim; /* STEP timer */
      uint32_t step_tim_clk;       /* Timer clock frequency in Hz*/
      uint32_t step_tim_channel;   /* Timer channel */
} nema_t;
```

```
typedef enum {
      NEMA_CW,
      NEMA_CCW
} nema_direction_t;
```

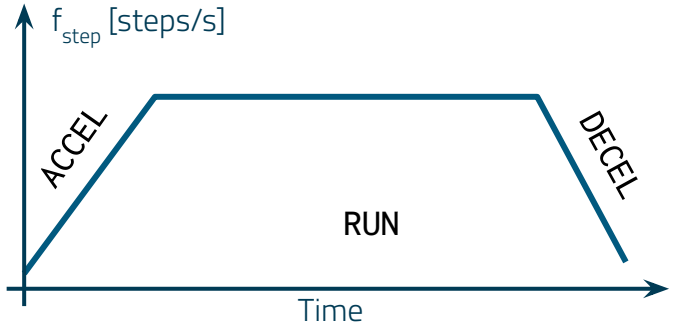
```
typedef struct {
      drv8825_microstep_t microsteps;
      /* M0 - M1 pins */
      GPIO_TypeDef *m0_port;
      uint16_t           m0_pin;
      GPIO_TypeDef *m1_port;
      uint16_t           m1_pin;
      GPIO_TypeDef *m2_port;
      uint16_t           m2_pin;
      /* Nsleep pin */
      GPIO_TypeDef *nsleep_port;
      uint16_t           nsleep_pin;
} drv8825_t;
```

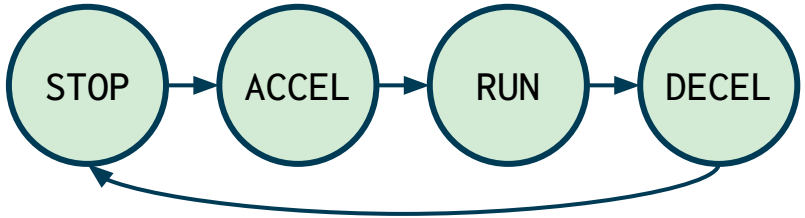
```
typedef enum {
      DRV8825_FULL_STEP=1,
      DRV8825_HALF_STEP=2,
      DRV8825_4_STEPS=4,
      DRV8825_8_STEPS=8,
      DRV8825_16_STEPS=16,
      DRV8825_32_STEPS=32
} drv8825_microstep_t;
```

Sketch of the structures

```
typedef struct {
      drv8825_t *drv;      /* Pointer to the driver */
      nema_t *nema;        /* Pointer to nema */
      float current_rpm;   /* Current rot. speed [rpm] */
      float target_rpm;    /* Desired rot. speed [rpm] */
      float accel_rpm_s;   /* Acceleration [rpm/s] */
      float decel_rpm_s;   /* Deceleration [rpm/s] */
      state_t state;       /* Motion state */
      uint8_t pwm_on;      /* PWM is running [boolean] */
} nema_drv8825_t;

typedef enum {
    NEMA_DRV8825_STOPPED,
    NEMA_DRV8825_ACCEL,
    NEMA_DRV8825_RUN,
    NEMA_DRV8825_DECEL
} nema_drv8825_motion_state_t;
```





References

[1] Jose I. Quinones, “Applying acceleration and deceleration profiles to bipolar stepper motors”, Texas Instruments application note

[2] STM32F446XX Reference manual

[3] DRV8825 datasheet